

AXI4-Lite Peripheral Subsystem — UVM Verification

SystemVerilog · UVM · SVA · Functional Coverage · Constrained Random

Ayman Shahapuri · Final-year ECE, Bengaluru · github.com/aym00n20005/axi4lite-uvm-verification

The design is deliberately small so the effort goes into the methodology. Two AXI4-Lite slaves — a register file with mixed RW/RO/W1C policies and a byte-addressable memory. Everything interesting is in the machinery built to check them.

BUILT

- Specification written and frozen **before** any RTL; corrected before it, never after
- **23 SVA assertions + 14 cover properties**, bound to the DUT so no assertion code sits in rtl/
- UVM agent with **five independent per-channel threads**, not a sequential FSM — AW and W are separate channels and a lockstep driver can never produce W-before-AW
- Passive monitor rebuilding transactions **from pins only**, with no route to the driver
- Scoreboard whose reference model is derived from the **specification**, not the RTL
- Coverage collector sampled from observation, reporting **missing bins by name**

MEASURED

Directed UVM	61 checks, 0 scoreboard mismatches
Random UVM	166 transactions, 0 mismatches
Non-UVM smoke	58 checks, two simulators, lint clean
Coverage	6 of 8 groups at 100%; every open bin justified in writing
Spec revisions	4, each traced to a test that fails under the old wording
Bug injections	4 of 6 detected, each by the mechanism the plan assigns it
Genuine bugs	3 found — all three in the testbench

TOOLS

Cadence Xcelium (UVM) · Verilator + Icarus (local RTL & SVA) · Make · Git. Aldec Riviera compiles UVM but cannot simulate it on the free tier — measured, not assumed.

WHAT THE TESTBENCH FOUND

A cover point that could not fail to be covered.

The property meant to prove the driver's threads run independently was satisfied by *any* multi-transaction run. It could never stay uncovered, so it could never warn. Qualifying it dropped the hit count from **10 to 1** on identical stimulus.

A correct, bound assertion sitting silently on a broken design.

I injected a real protocol defect and ran it three ways. Bound assertion: caught at the moment of violation. No assertion: caught twelve checks later as data corruption. Bound assertion with one stimulus case removed: **31 of 31 checks passed**. A checker constrains nothing if the stimulus never reaches the corner.

A protocol violation that every functional check passed.

Injected into my own driver: the bus was driven illegally on every write, all checks passed, and UVM_ERROR: 0. Only the assertion saw it. Functional correctness and protocol compliance are independent properties — demonstrated, not claimed.

Two contradictions in my own frozen specification.

Found while deriving the architecture on paper. One decode clause made a required error response unreachable; one status bit was defined so that reading it destroyed it. Both fixed in the document first, then the RTL.

Honest scope. In progress, and the plan runs to December. No interconnect yet, so DECERR and routed memory traffic are unreachable and recorded as such. No RAL model yet. Three coverage groups are listed as unmodelled rather than faked, because they need per-cycle timing the monitor does not record. Coverage is per-test and unmerged. Every number above came from a run; nothing is projected.